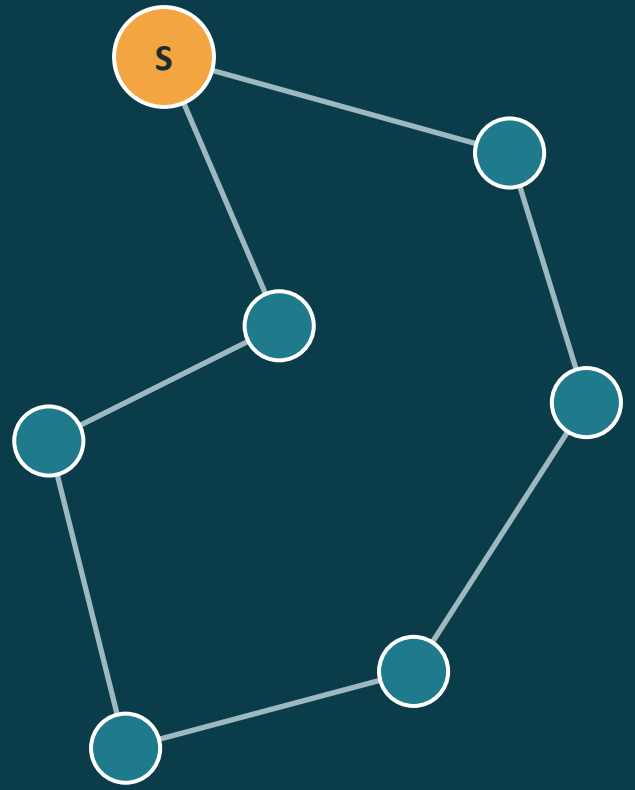
Travelling Salesman Problem

Finding the shortest route that visits every city exactly once



Presented by

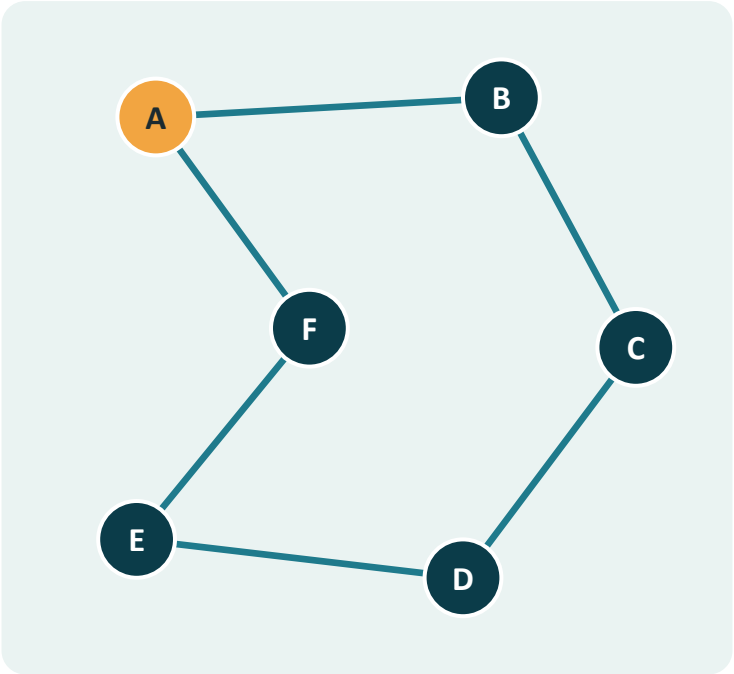
Tanvi Chakole

PRN: 25070521303

What is the Travelling Salesman Problem?

A salesman must visit a set of cities, visiting each city exactly once, and then return to the starting city.

1. **Goal**
   Minimise the total distance, cost or time of the tour
2. **Input**
   A list of cities and the distance between every pair
3. **Output**
   The best order in which to visit the cities



Formal Definition

1 Input

A complete weighted graph G = (V, E) with n cities and distance d(i, j) for every pair

2 Constraint

The tour is a Hamiltonian cycle: every city is visited exactly once, then return to start

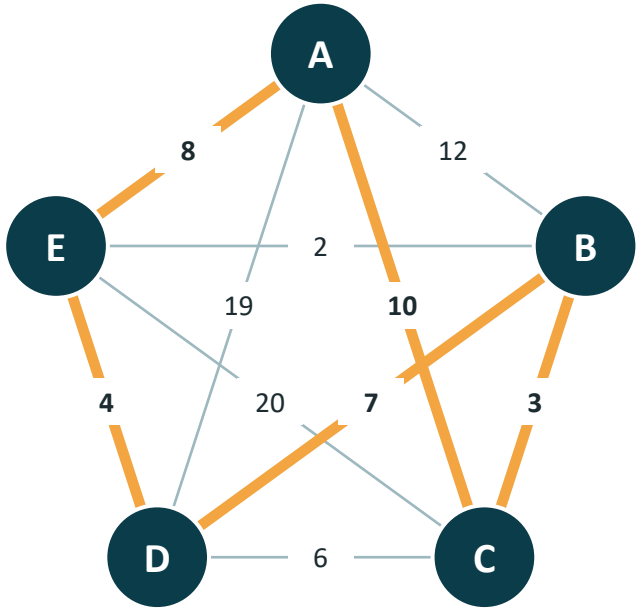
3 Objective

Choose the cycle whose total edge weight is the minimum possible

Minimise $L(\pi) = d(\pi_1, \pi_2) + d(\pi_2, \pi_3) + \ldots + d(\pi_n, \pi_1)$

where π is a permutation (ordering) of the n cities

Example: 5 Cities



Best tour found

A → C → B → D → E → A

32 total distance (highlighted route)

Checking all 12 distinct tours: the shortest is 32 and the longest is 59.

Why Is It So Hard?

Number of distinct tours for n cities = (n – 1)! / 2

n = 5	n = 10	n = 15	n = 20
12	**181,440**	**43.6**	**60.8**
tours	tours	billion tours	quadrillion tours

- Even at one billion tours per second, n = 20 would take about 2 years to check.
- TSP is NP-hard: no known algorithm solves every instance in polynomial time.

Exact Algorithms

Brute Force

O(n!)

Try every possible tour and keep the shortest. Only practical for very small n.

Held–Karp Dynamic Programming

$O(n^2 \cdot 2^n)$

Reuses best paths over subsets of cities. Far faster than brute force, but memory heavy.

Branch and Bound

Worst case exponential

Prunes partial tours whose lower bound already exceeds the best known tour.

Integer Linear Programming

Cutting planes

Used by the Concorde solver, which has solved instances with tens of thousands of cities.

Heuristics and Approximation

For large instances we trade a little optimality for a lot of speed.

Constructive and local search

Nearest Neighbour
Always go to the closest unvisited city; typically about 25% above optimal

2-opt / 3-opt
Swap edges to remove crossings and shorten the tour

Lin–Kernighan
Variable-depth edge exchanges; among the strongest practical heuristics

Approximation and metaheuristics

Christofides algorithm
Guarantees a tour at most 1.5× optimal for metric TSP

Genetic algorithm / Simulated annealing
Evolve or randomly perturb tours, escaping local minima

Ant colony optimisation
Virtual ants deposit pheromone on short routes

Common Variants

1 Symmetric TSP

Distance from A to B equals distance from B to A

2 Asymmetric TSP

Costs differ by direction, e.g. one-way streets

3 Metric TSP

Distances obey the triangle inequality

4 Multiple TSP (mTSP)

Several salesmen share the cities among them

5 TSP with Time Windows

Each city must be visited within a set time interval

6 Vehicle Routing Problem

Fleet of vehicles with capacity limits serving customers

Real-World Applications

1. **Delivery and logistics**
   Planning parcel and courier routes

2. **Circuit board drilling**
   Ordering drill holes to cut machine travel time

3. **DNA sequencing**
   Ordering fragments into a complete sequence

4. **Telescope scheduling**
   Sequencing observations to minimise slewing

5. **Warehouse order picking**
   Shortest path through shelves to collect items

6. **Trip planning**
   Visiting tourist spots in an efficient order

Conclusion

1. TSP is simple to state but NP-hard to solve in general.
2. Exact methods work for moderate sizes; heuristics handle large instances.
3. Its ideas power routing, manufacturing, science and logistics.

Thank You

Tanvi Chakole
PRN: 25070521303

Made with Claude